

East West University
Department of Computer Science and Engineering
Summer 2025
CSE207 – Data Structures, Section – 9
Course Instructor: Ahmed Abdal Shafi Rasel (AASR)

Transition from C to C++, Dynamic Memory Allocation, Pointers, Structures, Class

A. Transition from C to C++:

```
#include<stdio.h>

int main() {
    int n;
    scanf("%d", &n);
    printf("The number is %d\n", n);

    return 0;
}
```

code.c

```
#include<iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    cout << "The number is " << n << endl;

    return 0;
}
```

code.cpp

To learn C++ in depth, you can check the C++ documentation by clicking [here](#). If you want to write scanf and printf in c++, just add the header `#include<bits/stdc++.h>` instead of `#include<iostream>`.

B. Dynamic Memory Allocation:

Coding difference between C and C++

C	C++
<code>int *ptr = (int*)malloc(sizeof(int));</code>	<code>int *ptr = new int;</code>
<code>free(ptr);</code>	<code>delete ptr;</code>
<code>int *arr = (int*)malloc(sizeof(int) * 5);</code>	<code>int *arr = new int[5];</code>
<code>free(arr);</code>	<code>delete[] arr;</code>

C. Pointers:

```
#include <iostream>
using namespace std;

int main() {
    int a = 10;
    int *ptr = &a;
    cout << "Value: " << *ptr << endl;
    cout << "Address: " << ptr << endl;
    return 0;
}
```

Sample Output

Value: 10
Address: 0x6ef13ffc14

D. Structures:

```
struct Student {  
    int id;  
    char name[50];  
    float marks;  
};  
  
struct Student s1; // Explicit use of 'struct'
```

code.c

```
struct Student {  
    int id;  
    string name;  
    float marks;  
};  
  
Student s1; // No need to use 'struct' keyword
```

code.cpp

E. LinkedList using Self-Referenced Structs:

```
#include <iostream>  
using namespace std;  
  
struct Node {  
    int data;  
    Node* next;  
};  
  
void insert(Node*& head, int val) {  
    Node* newNode = new Node;  
    newNode->data = val;  
    newNode->next = NULL;  
    if (head == NULL) {  
        head = newNode;  
        return;  
    }  
    Node* temp = head;  
    while (temp->next != NULL) {  
        temp = temp->next;  
    }  
    temp->next = newNode;  
}  
  
void print(Node* head) {  
    Node* temp = head;  
    while (temp != NULL) {  
        cout << temp->data << "->";  
        temp = temp->next;  
    }  
    cout << "NULL\n";  
}  
  
int main() {  
    Node *head = NULL;  
    insert(head, 10);  
    insert(head, 20);  
    insert(head, 30);  
    print(head);  
    delete head;  
    return 0;  
}
```

Output

10→20→30→NULL

When using dynamic memory allocation for a structure in C++, the **new** keyword is needed. Since the allocated structure is accessed via a pointer, members of the structure should be assigned using → this sign.

F. LinkedList using Self-Referenced Class:

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;
    Node(int val) {
        data = val;
        next = NULL;
    }
};

class LinkedList {
private:
    Node* head;
public:
    LinkedList() {
        head = NULL;
    }
    void insert(int val) {
        Node* newNode = new Node(val);
        if (head == NULL) {
            head = newNode;
            return;
        }
        Node* temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
    void print() {
        Node* temp = head;
        while (temp != NULL) {
            cout << temp->data << "->";
            temp = temp->next;
        }
        cout << "NULL\n";
    }
};

int main() {
    LinkedList list;
    list.insert(10);
    list.insert(20);
    list.insert(30);
    list.print();
    return 0;
}
```

Output

10→20→30→NULL

Public: Members declared as **public:** are accessible from anywhere outside the class.

Private: Members declared as **private:** are hidden from outside access. They can only be accessed within the class itself.

Lab Task

1. Implement the Singly Linked-List (SLL) with its necessary functions.

Sample Input:

8
36 21 45 10 5 66 55 44

Sample Output:

36→21→45→10→5→66→55→44→NULL

2. Implement the Doubly Linked List with the following functions:

- ✓ **insert_at_end(data)**: Insert at the end.
- ✓ **insert_at_beginning(data)**: Insert at the beginning.
- ✓ **delete_from_end()**: Delete from the end.
- ✓ **delete_from_beginning()**: Delete from the beginning.
- ✓ **display_forward()**: Print list from start to end.
- ✓ **display_backward()**: Print list from end to start.
- ✓ **search(data)**: Search for data in the list.
- ✓ **delete(data)**: Delete node containing specific data

Handle empty list cases for insertions, deletions, and searches.